LAGOON implementation plan

Oct 1, 2026 · @Hans Bihs

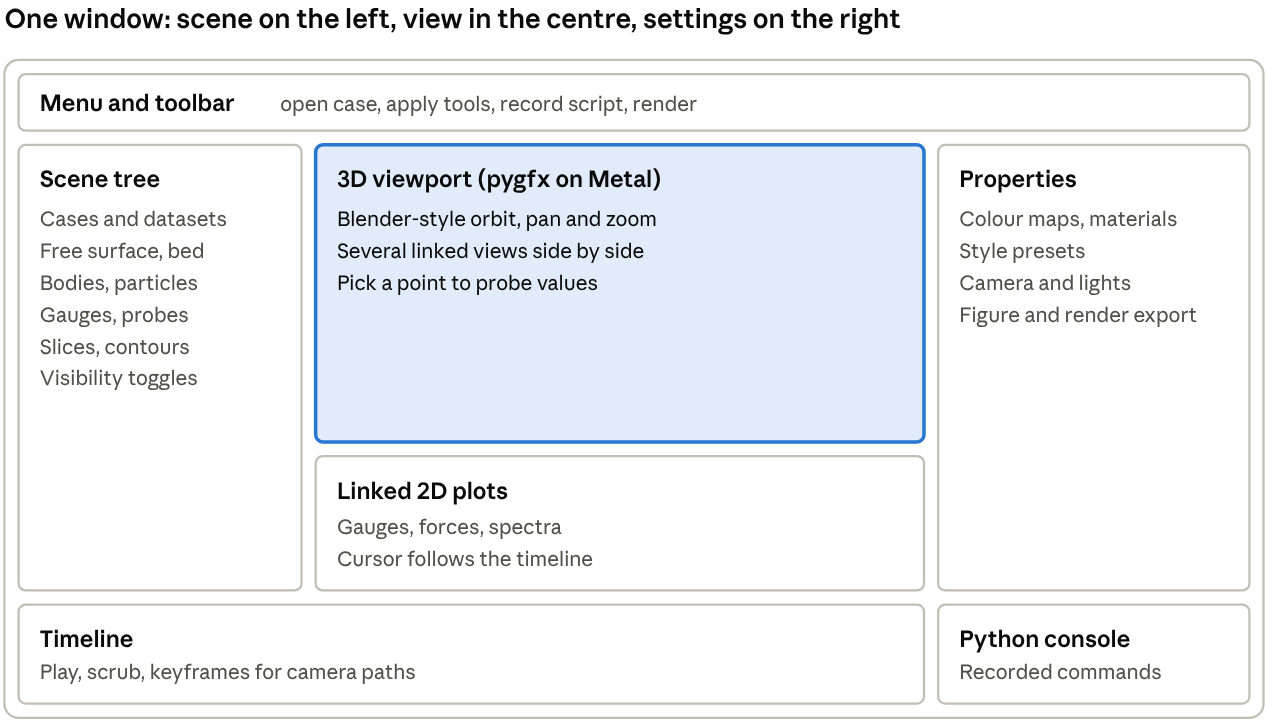
Approach

LAGOON is built GUI-first: the first release is already a desktop application in the style of ParaView and Blender, with a 3D viewport, scene tree, properties panel and timeline. Underneath, a strict split keeps all logic in a GUI-free core, so everything the application does can also run from scripts and headless on clusters.

- **Viewer before format.** The first milestone opens existing REEF3D VTU/PVTU output, so the application is useful on real runs from the start. The new LAGOON data format follows once the viewer shows what the format needs.
- **Blender's interaction model, ParaView's data model.** Viewport navigation, scene tree, properties panel and timeline follow Blender. Time series, multiple linked views and data inspection follow ParaView, but organised by task instead of by filter.
- **Every action is a command.** Each GUI action calls the core through a command object. This gives undo/redo, saved sessions, and a script recorder that turns a GUI session into a readable Python script.
- **Each phase ends usable.** Every phase delivers something the group can use in daily work, so feedback comes early and the project never depends on a distant big release.

Main window

The main window follows Blender's layout: what is in the scene on the left, the view in the centre, settings for the selected item on the right, and time along the bottom.



LAGOON main window · wireframe

The viewport is the heart of the application; the plot panel below it shows time series that stay in sync with the timeline and the 3D view.

- **Scene tree:** every loaded case and everything derived from it (surfaces, bodies, slices, gauges) with visibility toggles, like Blender's outliner. It replaces ParaView's pipeline browser with domain items instead of filters.
- **Properties:** settings for the selected item only, grouped by task: colour map, material, style preset, camera, export.
- **Timeline:** play and scrub through time; keyframes set camera paths for animations.
- **Python console:** shows each command the GUI runs, so users learn the scripting interface by watching.

Code architecture

The code is split into four layers with one-way dependencies, checked automatically in CI so the split cannot erode.

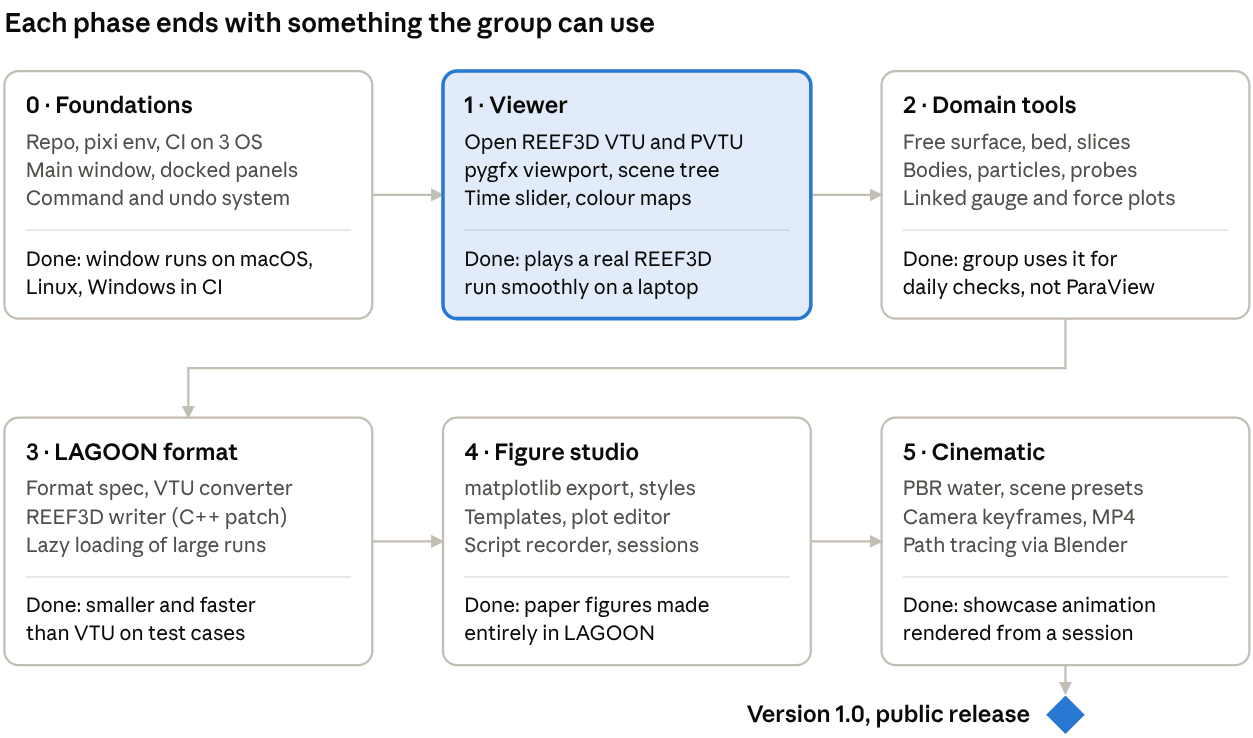
Package	Holds	May import
`lagoon.core`	Data model, readers, LAGOON format, analysis, scene and figure descriptions, commands	NumPy, xarray, zarr, VTK (headless)
`lagoon.render`	pygfx scenes, matplotlib figures, offscreen and path-traced rendering	`core`, pygfx, matplotlib
`lagoon.gui`	Qt windows, panels, widgets, event handling	`core`, `render`, PySide6
`lagoon.plugins`	Readers and tools for other codes and data	`core`, optionally `render`

Rules that keep the core usable without a GUI:

- `core` never imports Qt or pygfx. An import-linter contract in CI fails the build if it does.
- **Commands:** every user action (load case, add slice, set colour map, move camera) is a command object in `core`. The GUI only creates and runs commands. This gives undo/redo, and the command log is what the Python console shows and the script recorder saves.
- **Scene description:** the state of a session (loaded cases, scene items, styles, cameras, keyframes) is a plain, serialisable description in `core`. Saving a session writes it to a file; the same file opens in the GUI or renders headless on a cluster.
- **Threading:** loading and filtering run in background workers, so the window never freezes. Rendering stays on the main thread, as Qt and wgpu require.
- **Design test:** every figure or animation made in the GUI must be reproducible from a saved session on a machine without a screen. This is an automated test from the first phase on.

Roadmap

Six phases lead to version 1.0; phase 1 is the first release the group can use on real REEF3D runs.



LAGOON roadmap · 6 phases with done-criteria

Each phase ends with a done-criterion that is checked in practice, not just a list of finished features. Phases 3 to 5 can partly overlap once the viewer and domain tools are stable.

Phases in detail

Effort figures are rough estimates for one full-time developer; together they add up to roughly one and a half to two years to version 1.0.

Phase 0: Foundations (about 1 month)

- Repository `REEF3D/LAGOON`, license, `pyproject.toml`, pixi environment from conda-forge.
- GitHub Actions running tests on macOS, Linux and Windows; import-linter contract for the layer rules.
- Main window with dockable panels (scene tree, viewport, properties, timeline, console), empty but resizable and saved between sessions.
- Command system with undo/redo and a command log shown in the console.
- **Done when** the empty application starts and passes CI on all three platforms.

Phase 1: Viewer (2 to 3 months)

- Readers for REEF3D VTU/PVTU output from CFD, NHFLOW and FNPF, including merging MPI pieces and finding time steps.
- pygfx viewport with Blender-style navigation, surfaces coloured by a chosen variable, colour maps and colour bars.
- Scene tree with visibility, properties panel for the selected item, timeline with play and scrub.
- Loading in background workers with progress, and caching of recently viewed time steps.
- **Done when** a real REEF3D run, for example the internal-wave seamount case, plays smoothly on a laptop.

Phase 2: Domain tools (3 to 4 months)

- Task-level items: free surface, bathymetry, slices, iso-surfaces and contours, vectors and streamlines (VTK headless).
- Bodies and particles: 6DOF bodies from STL and VTP output, DEM particles, ice floes and rod trees as instanced meshes.
- Gauges, probes and forces in the linked 2D plot panel, with a cursor tied to the timeline; click in the viewport to probe values.
- Several linked 3D views, for example top view and perspective side by side.
- **Done when** the group uses LAGOON instead of ParaView for daily checks of runs.

Phase 3: LAGOON format (2 to 3 months)

- Written specification of the structured Zarr layout for Cartesian and σ-grids, bodies, gauges and metadata.
- VTU-to-LAGOON converter and Python reader with lazy, chunked loading.
- REEF3D C++ writer delivered as a patch to hans_dev, one chunk set per MPI rank.
- ParaView reader plugin so other tools can still open the data.
- **Done when** the format is clearly smaller and faster to load than VTU on the reference cases, and large runs open instantly.

Phase 4: Figure studio (3 to 4 months)

- Publication export through matplotlib: 2D plots, and 3D views composited with axes, colour bars, labels and LaTeX equations.

- House styles (REEF3D palette, fonts, line widths) and figure templates for validation plots, time series and spectra.
- DataGraph-style plot editor: drag variables onto axes, edit styles live, save as a template.
- Saved sessions and a script recorder that writes a readable Python script from the command log.
- **Done when** the figures of a group paper are made entirely in LAGOON.

Phase 5: Cinematic rendering (3 to 4 months)

- Physically based materials for water, sand, rock, concrete and steel; scene presets such as open ocean, wave flume and underwater view.
- Camera keyframes on the timeline, smooth camera paths, MP4 export.
- Path-traced output by sending the scene to headless Blender as a separate process.
- **Done when** a showcase animation is rendered directly from a saved session, without opening Blender.

Version 1.0

Public release on pip and conda-forge, documentation with example cases, a gallery from the group's studies, and a JOSS paper.

Technology choices

The stack is entirely open source and runs on macOS, Linux and Windows.

Area	Choice	Why
Language	Python, C++ only for the REEF3D writer	Fast development; the heavy numerics already live in compiled libraries
GUI	PySide6 (Qt 6)	Mature docking, native look on all platforms, LGPL license
3D rendering	pygfx on wgpu	Modern GPU rendering through Metal, Vulkan or Direct3D 12; embeds in Qt

Area	Choice	Why
Interactive 2D plots	fastplotlib (on pygfx)	Fast enough for live, linked plots; same rendering engine as the viewport
Publication figures	matplotlib	Best typography and LaTeX support; house styles
Readers and filters	VTK, headless	Proven readers for VTU/PVTU and filters for slices, contours, streamlines
Data model and format	xarray, Zarr	Labelled arrays, lazy chunked loading, simple layout for parallel writing
Path tracing	Headless Blender (Cycles) as a separate process	Film-quality output without embedding Blender; keeps LAGOON's license free
Environment	pixi (conda-forge)	Locked, identical environments for every contributor and CI
Testing	pytest, pytest-qt, pytest-mpl	Core logic, GUI behaviour and figure output each tested automatically
IDE	VS Code	Python and C++ in one place, git, Jupyter, remote work on clusters

Interactive plots in the GUI use fastplotlib for speed, while the exported figure is always redrawn with matplotlib from the same figure description, so the screen and the paper show the same content.

Risks and open decisions

The biggest technical risk is the young pygfx stack; the biggest project risk is GUI work growing faster than the core.

Risk	Mitigation
pygfx/wgpu is young and may lack features or have platform bugs	Prototype the viewport on real REEF3D data in phase 1; keep a VTK render path as fallback
GUI polish absorbs time while the data model still changes	Minimal panels until phase 2; no layout polishing before the data model settles
Logic creeps into the GUI	import-linter contract, commands for every action, headless session test in CI
Large runs too slow to load from VTU	Background loading and caching in phase 1; LAGOON format in phase 3 solves it properly
Too few developers	Clear first issues for students, documentation from phase 1, early use by the group

Open decisions:

- [ ] License: BSD-3-Clause or MIT (recommended) versus GPL, before the first outside contribution.
- [ ] PyPI package name, for example `lagoon-viz`, and whether to reserve it now.
- [ ] Who develops: a research software engineer, a PhD project, or students with a core maintainer.
- [ ] Whether phase 3 (format) should start in parallel with phase 2 if a second developer is available.

First steps

The first weeks set up the foundations and prove the riskiest piece, the pygfx viewport inside Qt, on real data.

- [ ] Create the `REEF3D/LAGOON` repository, choose the license, reserve the PyPI and conda-forge names.
- [ ] Set up the package layout (`core`, `render`, `gui`, `plugins`), the pixi environment and the VS Code workspace.

- [ ] Add GitHub Actions for macOS, Linux and Windows, with pytest and the import-linter contract.
- [ ] Build the empty main window with docked panels and the command system with undo/redo.
- [ ] Spike: embed a pygfx canvas in the Qt viewport and show one free surface from an existing REEF3D VTU file, with a time slider.
- [ ] Collect two or three reference cases (for example the internal-wave seamount and the DEM breakwater) as test data for phases 1 and 2.